



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 9: Integrating CI/CD with Machine Learning Pipelines

Dr Pedro Machado pedro.machado@ntu.ac.uk



Nottingham Trent
University

Department of Computer Science

Lecture Outline

In this lecture will cover:

- Foundations of DevOps, CI/CD, and the evolution toward MLOps
- Reference MLOps architectures, components, and lifecycle processes
- CI integration strategies for different ML paradigms:
 - Clustering (unsupervised validation, stability testing)
 - Association Rule Mining (rule regression, support drift)
 - Privacy-Preserving Mining (federated CI, differential

This lecture contributes to the following learning outcomes:

MLO2: Critically evaluate the strengths and weaknesses of different computational intelligence techniques for solving complex problems.

MLO4: Understand the ethical implications of using computational intelligence technologies and the potential impact on society.

MLO5: Apply computational intelligence techniques to solve real-world problems, demonstrating practical problem-solving skills.

MLO6: Develop proficiency in programming and using computational tools for implementing and evaluating intelligent systems.

MLO8: Communicate effectively the concepts, methodologies, and results of computational intelligence research to both technical and non-technical audiences.

- privacy validation)
- Hybrid CI-ML systems combining software engineering and ML lifecycle automation
- Continuous Monitoring, retraining, and deployment strategies
- CI considerations for GenAI and foundation model pipelines (prompt testing, RAG validation, safety auditing)
- Cross-cutting themes: reproducibility, governance, maturity models, open research challenges

Why CI/CD for ML ?

- High production failure rate: Many ML models never reach stable deployment
- Operational complexity: Integration, monitoring, retraining challenges
- Data & concept drift: Models degrade without continuous validation
- Reproducibility needs: Versioning of code, data, and models
- Lack of standardised metrics: Limited formal evaluation frameworks



DevOps Recap



CI/CD enabling
automated build, test,
and release cycles



Automation-first mindset
to reduce manual
intervention



Version control across
code and configuration



Reproducible builds and
consistent environments



Cultural shift toward
collaboration between
development and
operations

What Changes with ML?

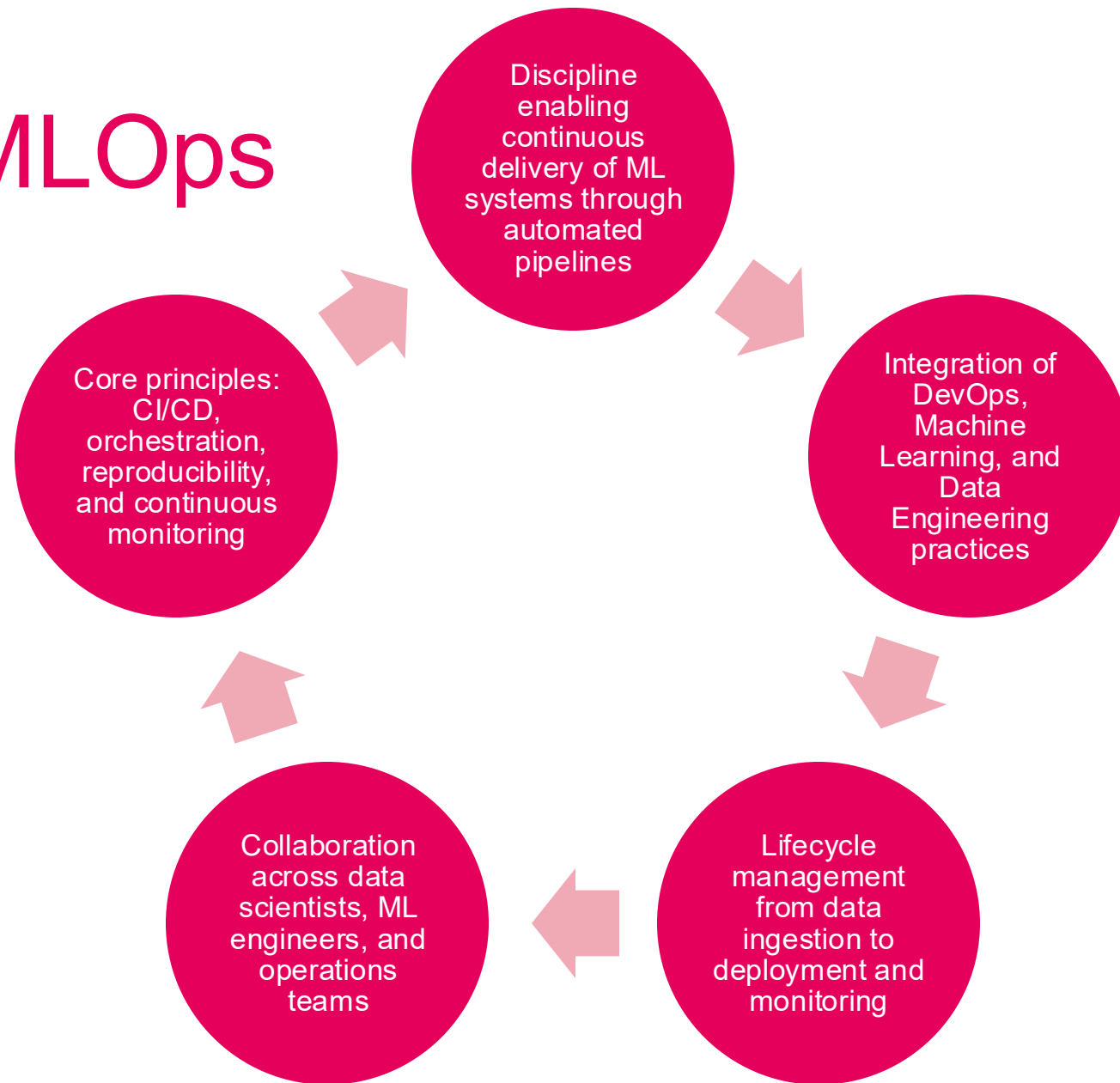
Data as a first-class artifact
alongside code

Model drift requiring continuous
validation and retraining

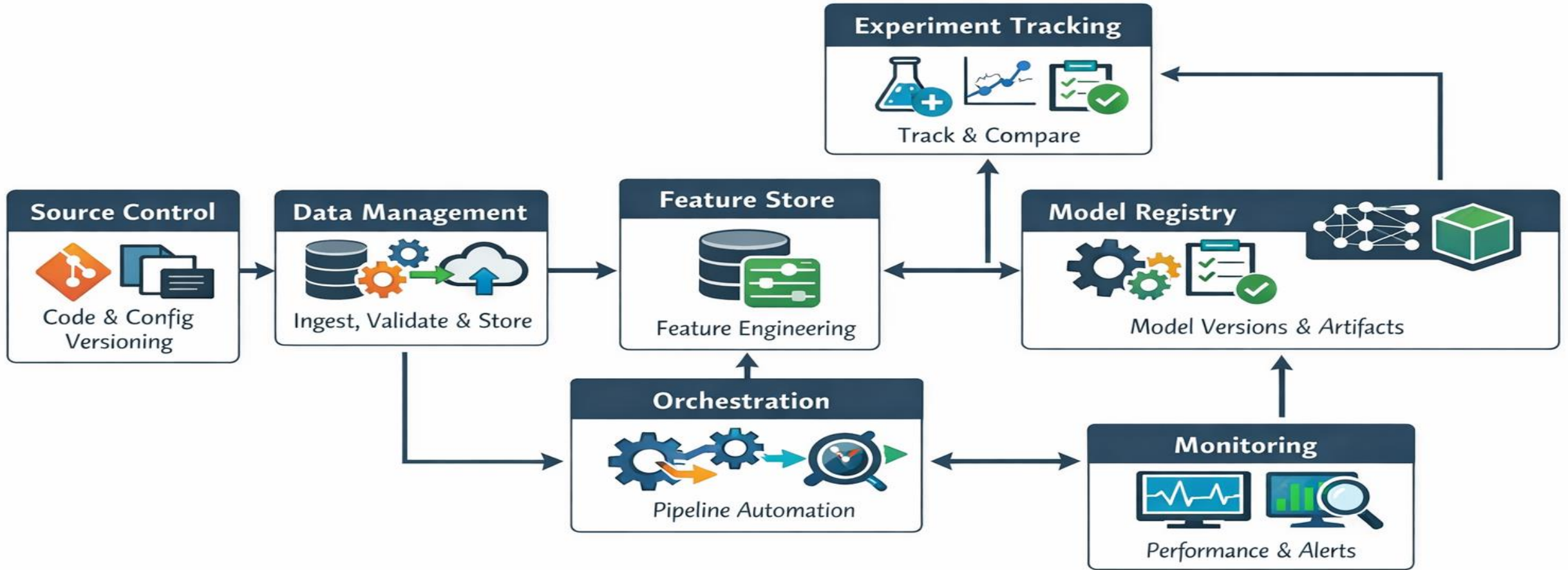
Experiment tracking for systematic
comparison

Provenance and lineage across
data, features, and models

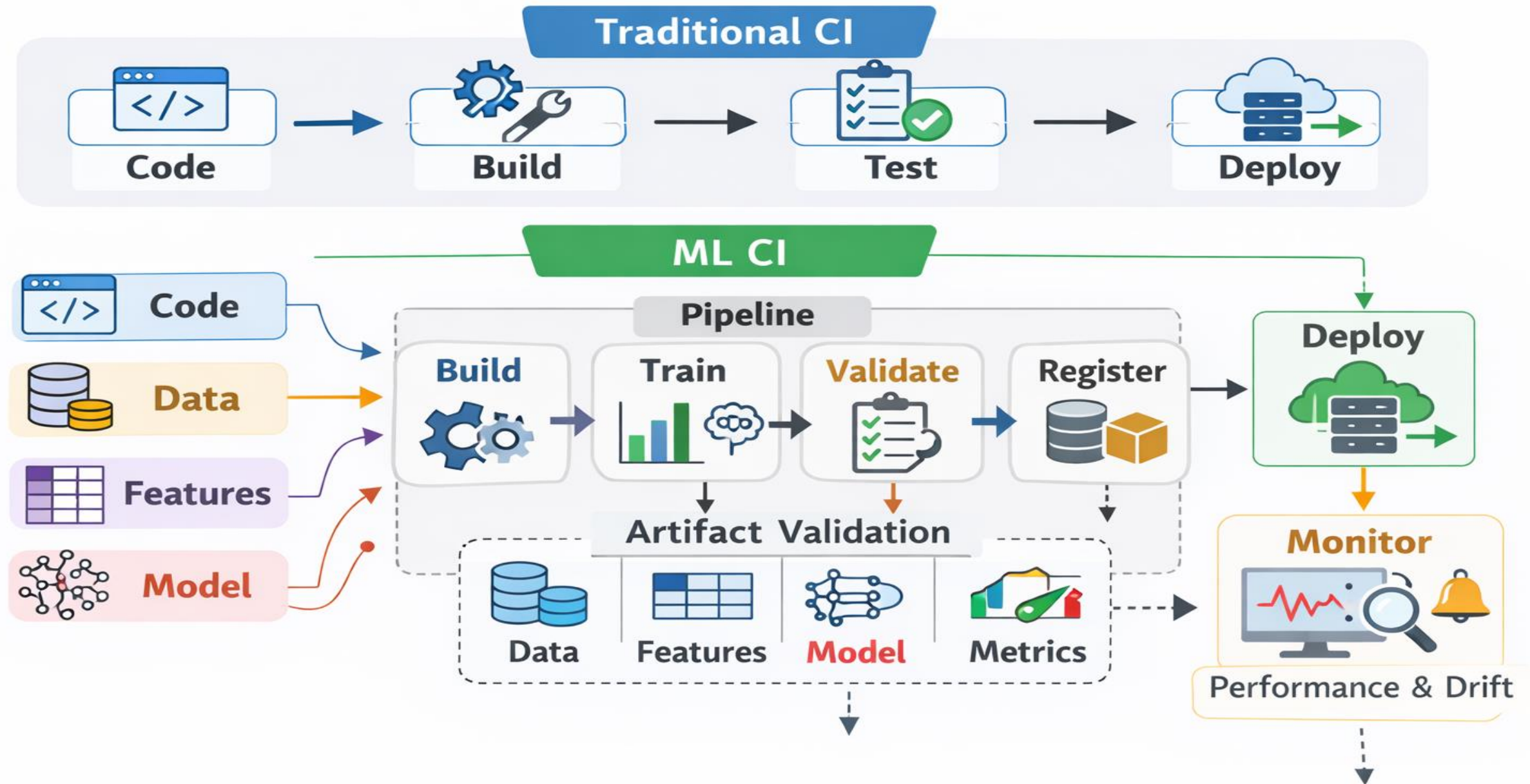
Defining MLOps



Reference MLOps Architecture



CI in Traditional vs ML Pipelines



CI/CD for ML: Core Requirements

Versioning across code, data, features, and models

Automated testing of models and data pipelines

Reproducible training and deployment environments

Artifact validation beyond compiled binaries

Continuous retraining and integration of updated models

Nature of Clustering in Production

01

Unsupervised learning without ground-truth labels

02

Limited use of traditional accuracy-based CI tests

03

Cluster quality assessed via internal metrics

04

Drift detection more complex and indirect

05

Greater sensitivity to feature distribution changes

CI Challenges in Clustering

- Sensitivity to hyperparameters and configuration changes
- Non-deterministic initialisation leading to inconsistent outputs
- Ambiguity in selecting and interpreting internal metrics
- Lack of external ground truth for regression testing
- Difficulty defining stable acceptance thresholds



CI Testing/CD Strategy for Clustering

Use of deterministic seeds to reduce randomness

Stability testing across multiple runs and splits

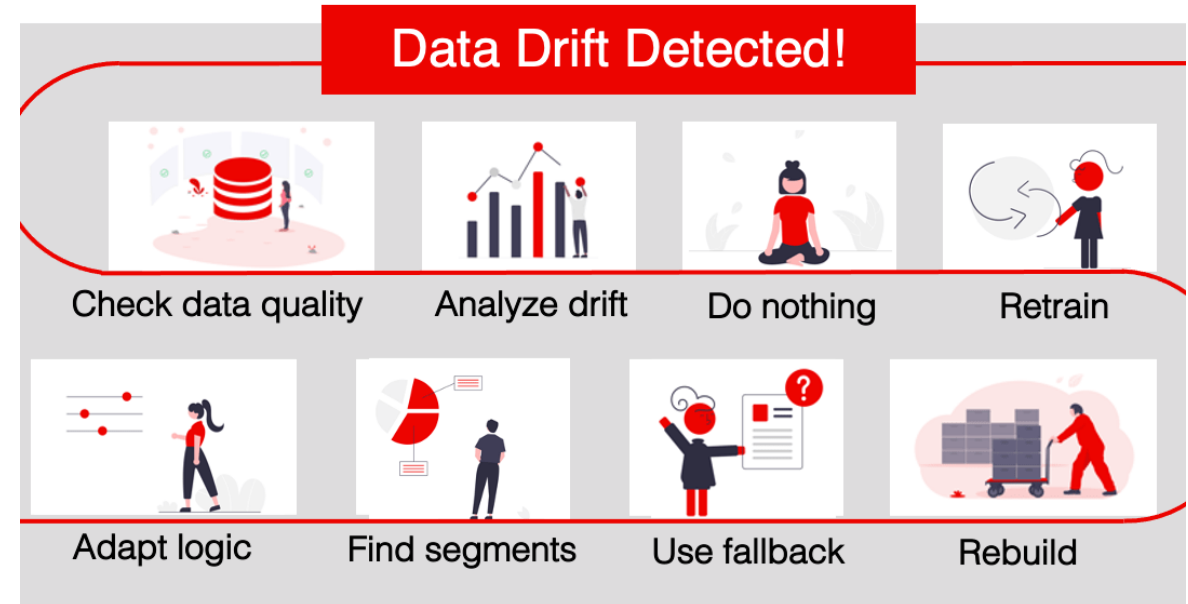
Comparison of cluster assignments over time

Distributional consistency checks on features

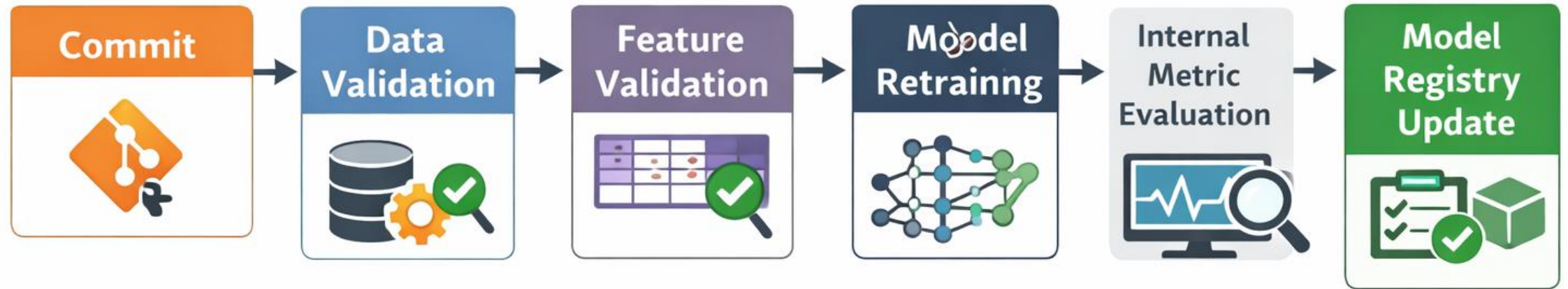
Threshold-based validation of internal metrics

Data Drift Detection for Clustering

- Monitoring of feature distribution changes over time
- Tracking cluster centroid movement across retraining cycles
- Distance-based comparison of cluster structures
- KL-divergence or statistical distance alerts. Kullback-Leibler (KL) divergence is a statistical measure that quantifies how one probability distribution differs from another reference distribution.
- Automated triggers for retraining when thresholds are exceeded

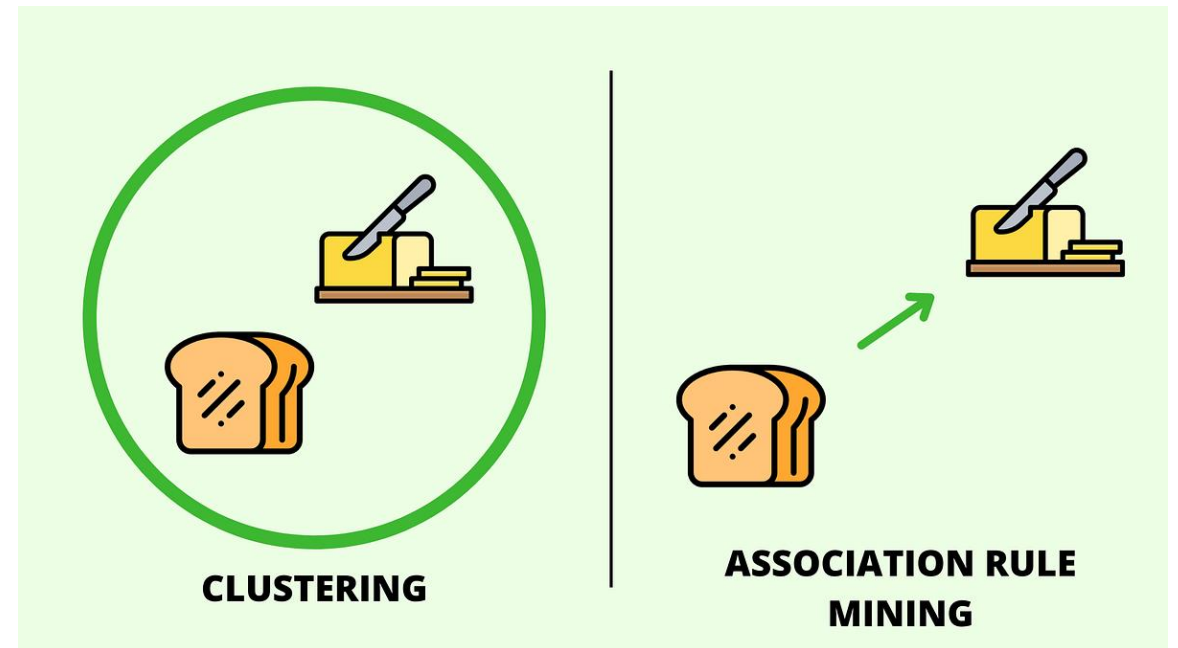


CI Pipeline Example (Clustering)



Association Rule Mining Overview

- Discovery of frequent itemsets from transactional data
- Generation of rules using support, confidence, and lift
- Identification of co-occurrence patterns and dependencies
- Commonly applied in retail, recommender, and basket analysis
- Scalable to large transactional and event-based datasets



CI/CD Challenges for Rule Mining



High sensitivity to data volume and distribution changes

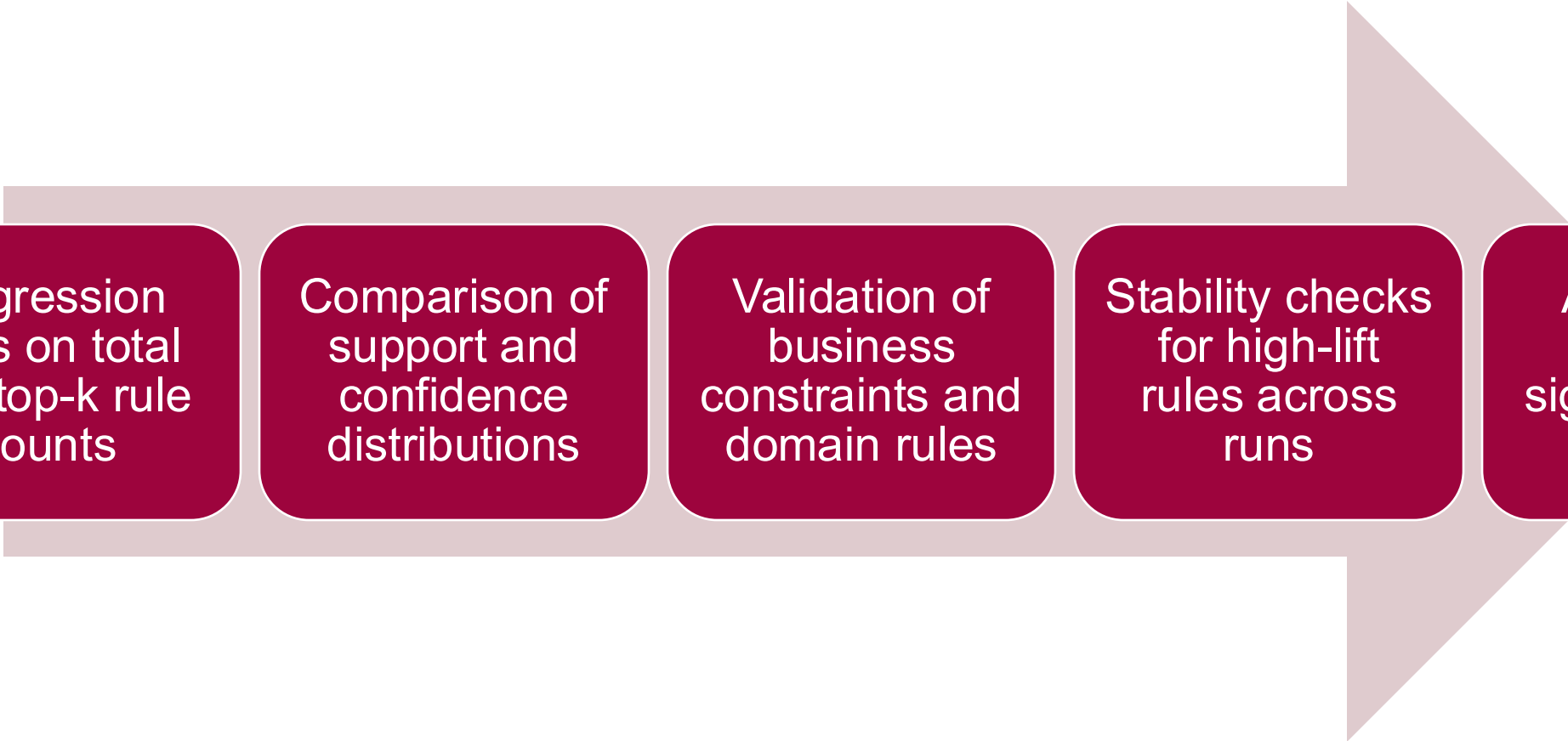
Risk of rule explosion with large itemsets

Instability caused by small threshold adjustments

Difficulty defining regression baselines for rules

Managing computational cost in automated pipelines

CI/CD Testing for Association Rules



Regression tests on total and top-k rule counts

Comparison of support and confidence distributions

Validation of business constraints and domain rules

Stability checks for high-lift rules across runs

Automated alerts for significant rule drift

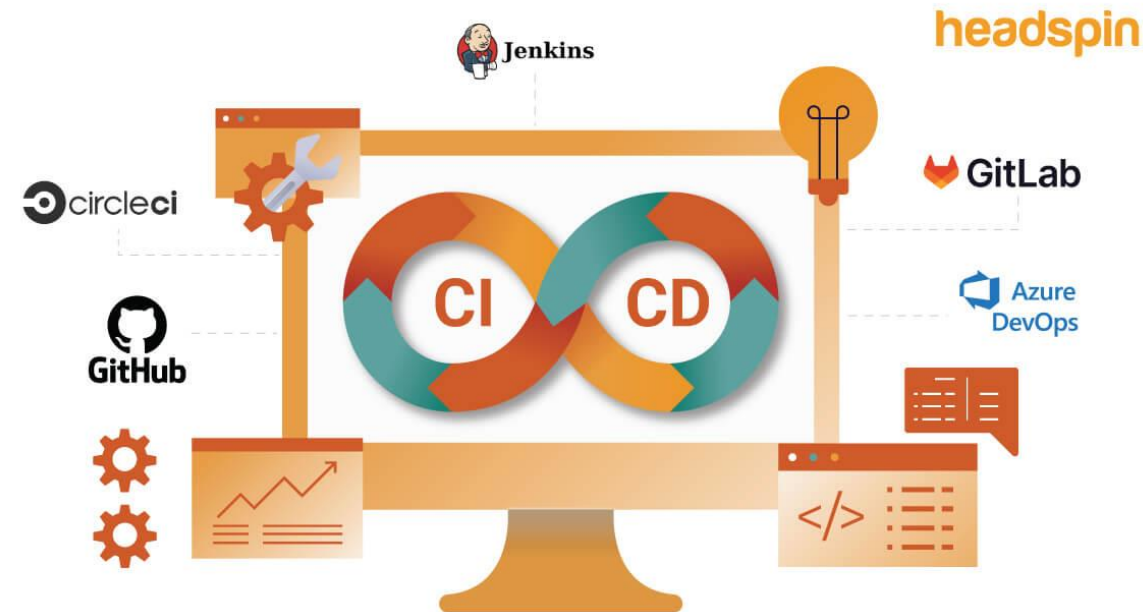
CI/CD Validation Metrics

- Stability of top-k rules across retraining cycles
- Drift analysis in support and confidence values
- Monitoring changes in lift and rule strength
- Business-rule compliance and constraint validation
- Threshold-based alerts for significant deviations



CI/CD Automation Strategy

- Automated optimisation of support and confidence thresholds
- Rule pruning and redundancy reduction checks
- Scheduled retraining with regression safeguards
- Reproducibility validation across environments
- Automated reporting and audit logging of rule changes



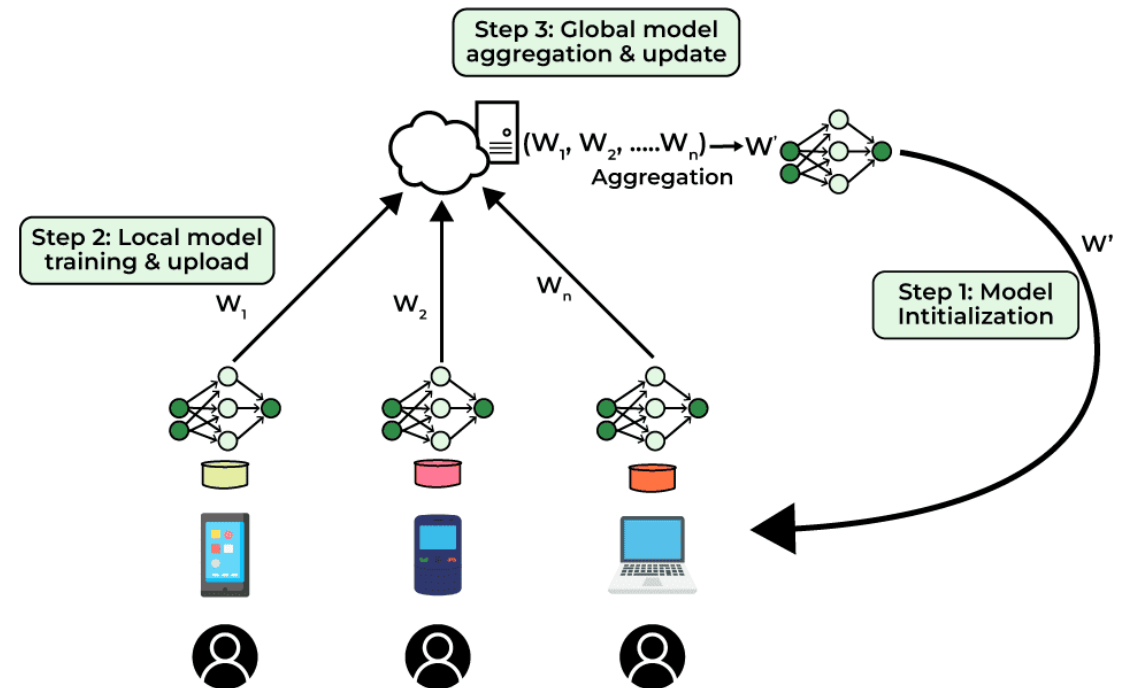
Why Privacy Changes CI/CD

- Federated learning with distributed data and decentralised training
- Differential privacy introducing controlled noise into model updates
- Secure multi-party computation without raw data sharing
- Restricted access to training data in CI pipelines
- Additional validation for privacy guarantees and compliance



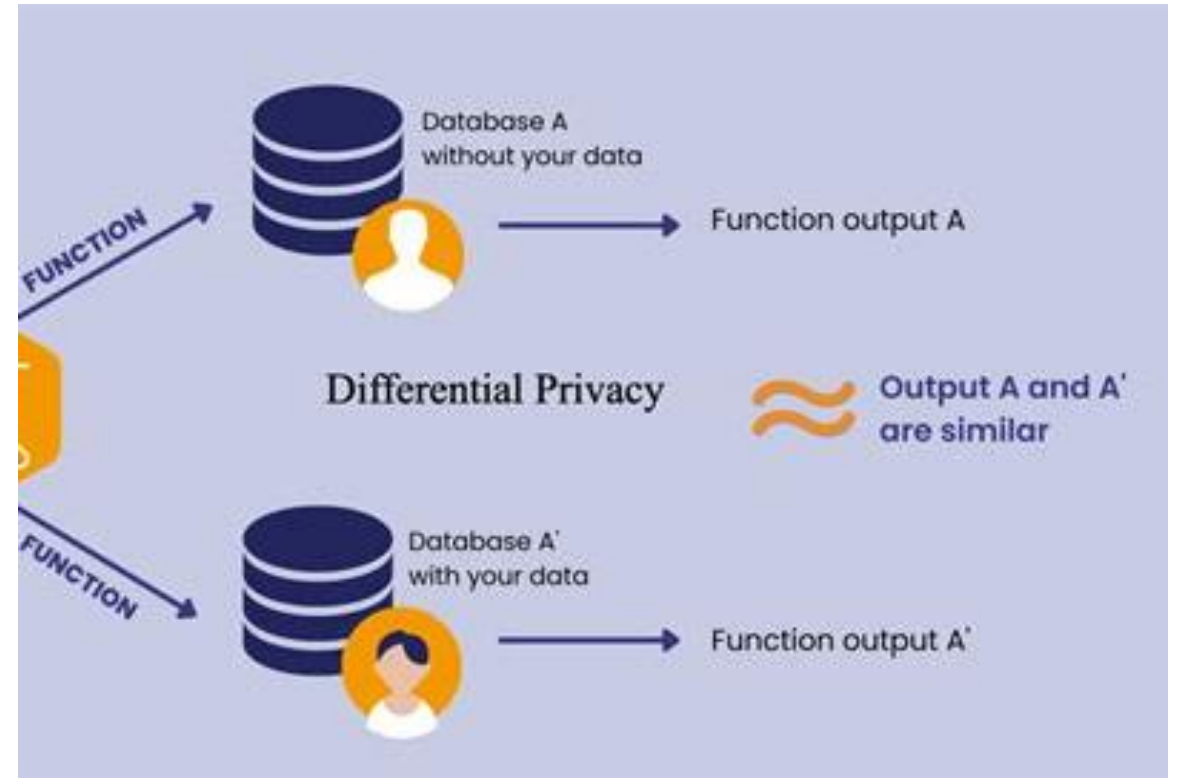
CI/CD in Federated Settings

- Validation of model aggregation across distributed nodes
- Secure parameter exchange and encrypted communication
- Consistency checks on local model updates
- Evaluation without direct access to raw training data
- Automated monitoring of convergence and stability



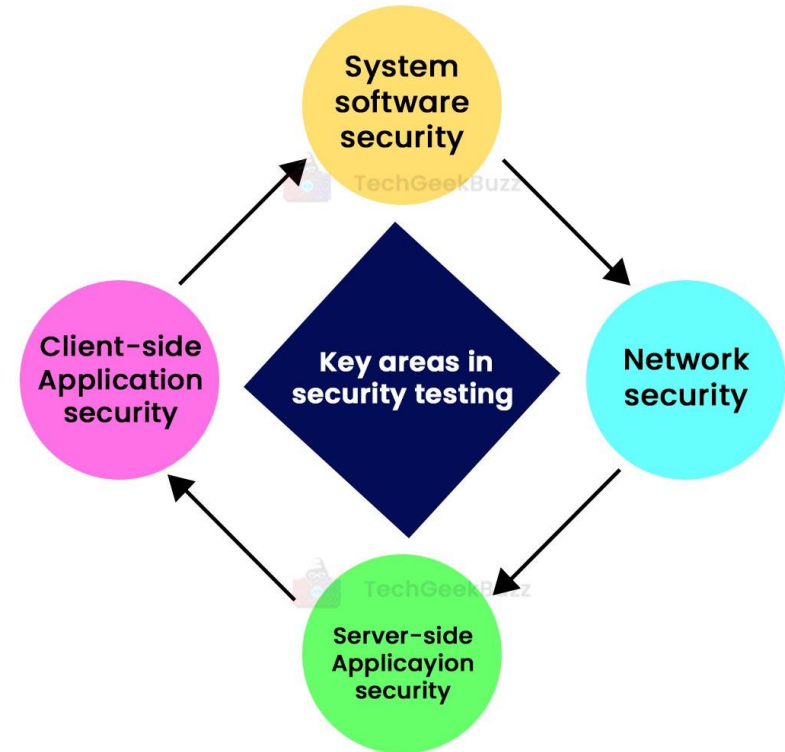
CI/CD with Differential Privacy

- Validation of noise injection mechanisms
- Monitoring and tracking of privacy budget consumption
- Testing trade-offs between privacy and model utility
- Regression checks on performance under privacy constraints
- Automated alerts for privacy threshold violations



Security Testing in CI/CD

- Testing resistance to model inversion attacks
- Membership inference vulnerability assessments
- Adversarial robustness evaluation under perturbed inputs
- Automated security regression tests in pipelines
- Monitoring for anomalous model behaviour



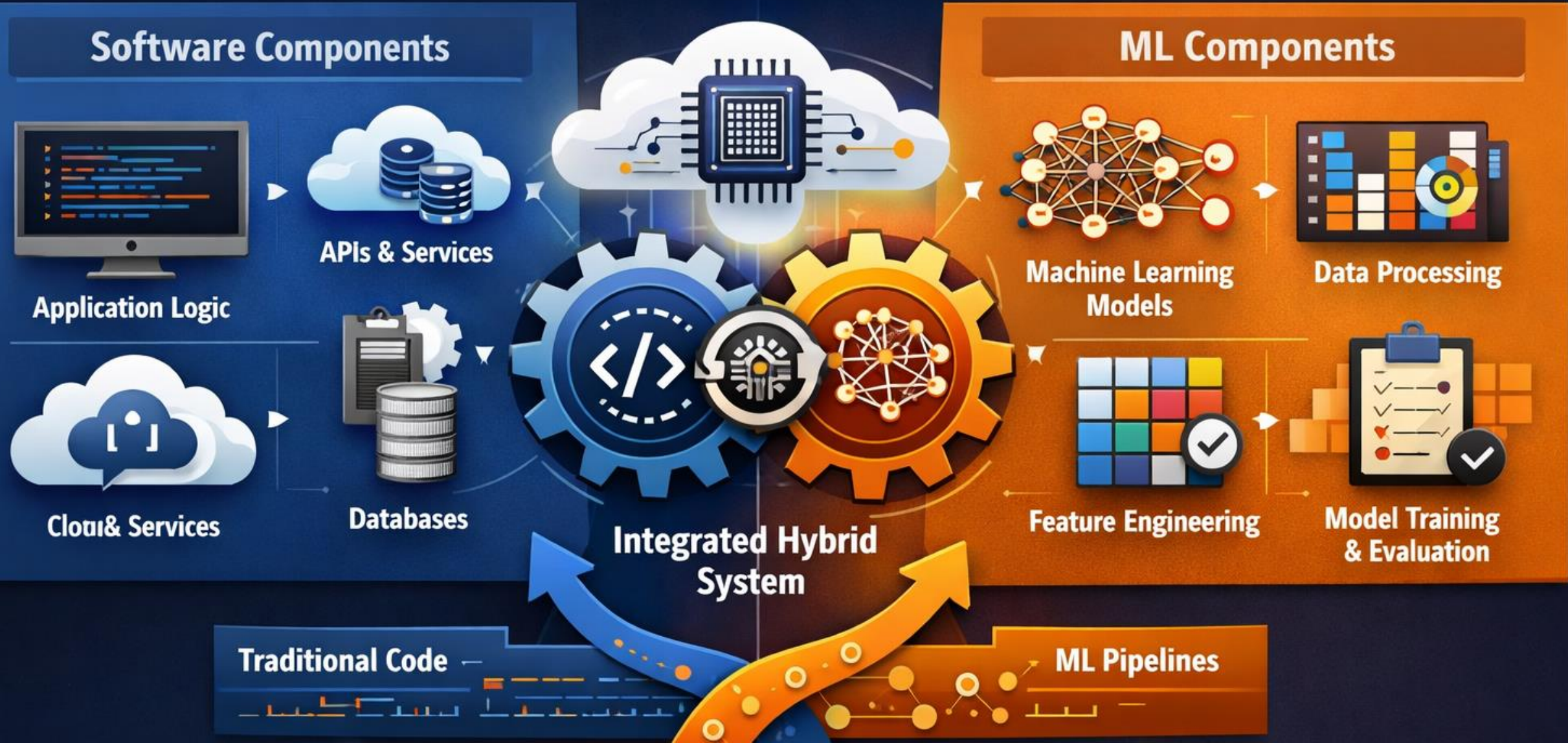
Governance & Compliance in CI/CD

- Comprehensive audit trails for pipeline activities
- Provenance and lineage tracking across data and models
- Version-controlled artifacts for reproducibility
- Access control and role-based permissions
- Compliance validation aligned with regulatory requirements



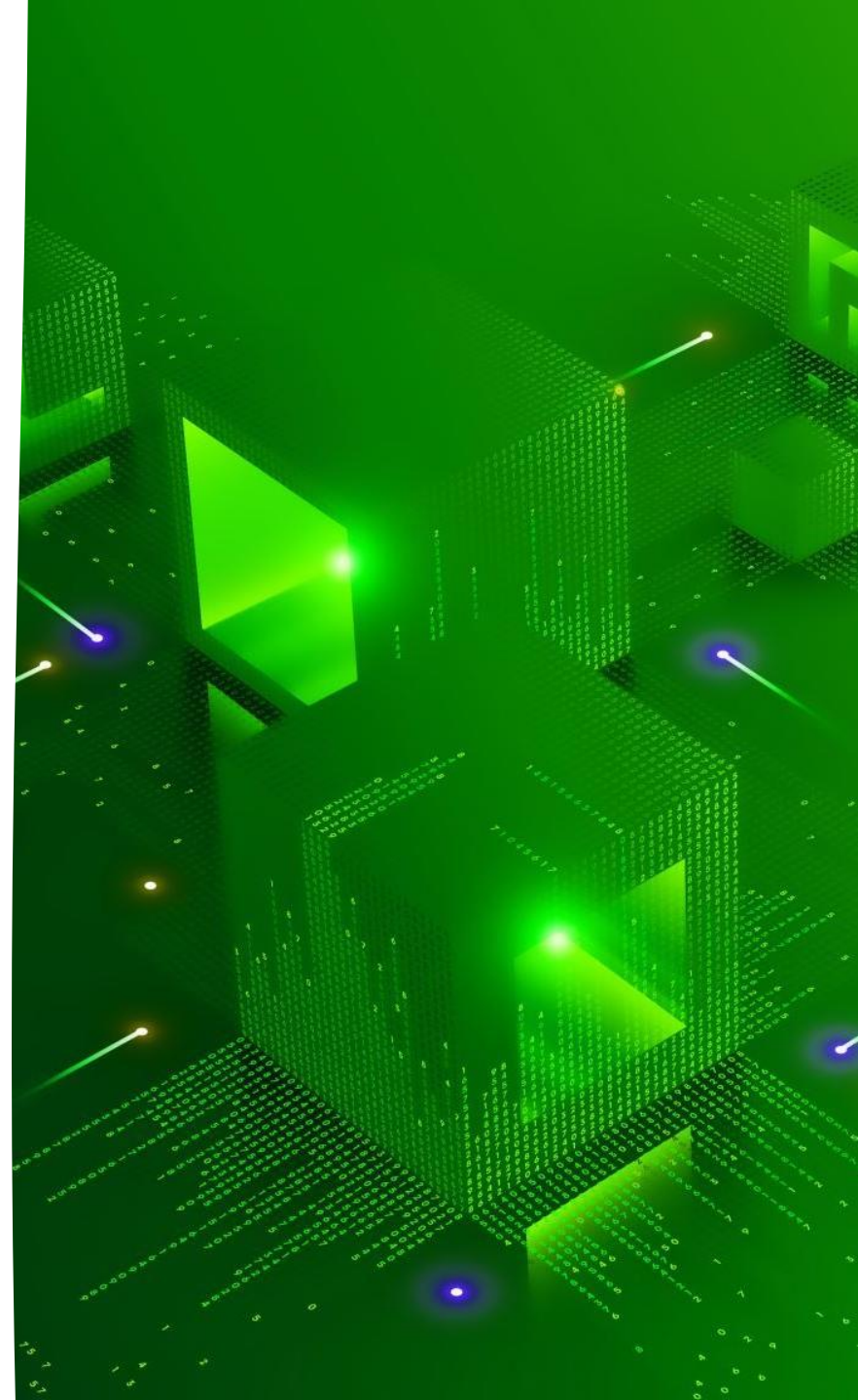
Hybrid Systems Definition

Tightly Coupled Software + ML Components



Integration Testing Challenges

- Schema mismatches between data producers and consumers
- Errors in feature transformation or preprocessing logic
- Inconsistent feature definitions across environments
- Latency and performance constraints in real-time systems
- Coupling issues between software components and ML models



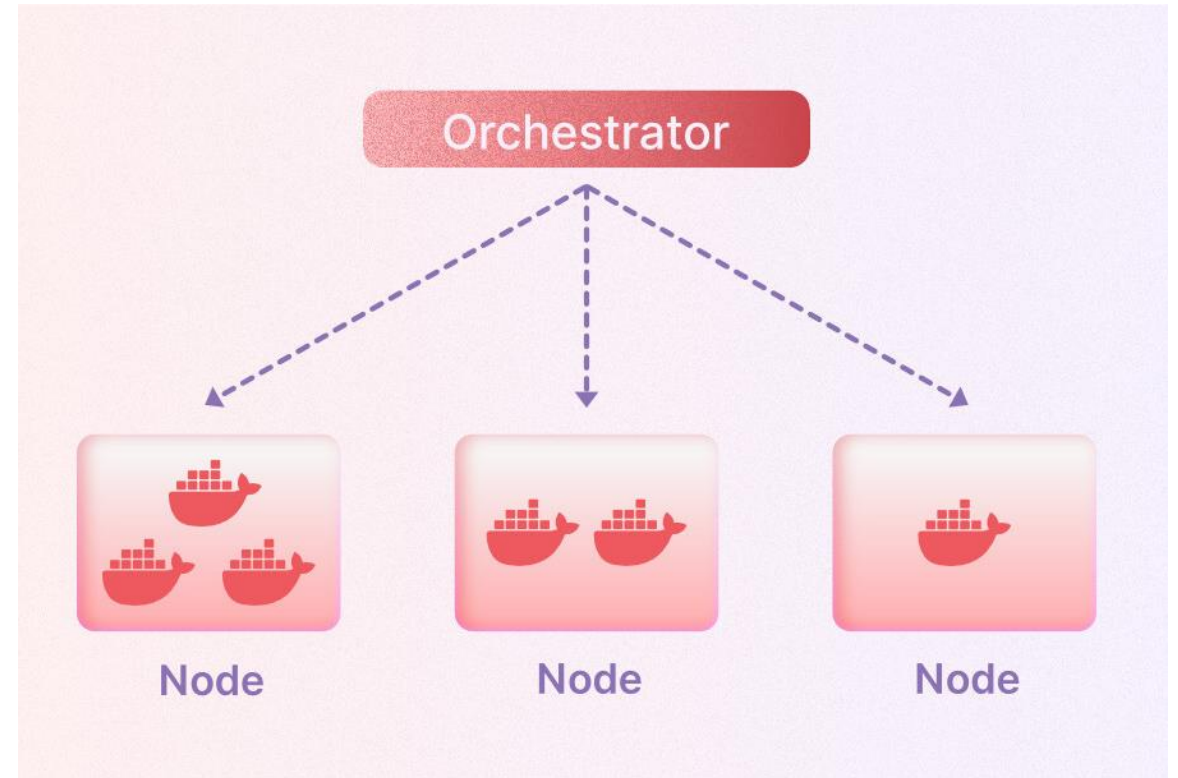
CI/CD and Continuous Training (CT)

- Continuous Integration for frequent code and pipeline updates
- Continuous Delivery for automated and reliable deployments
- Continuous Training to retrain models on updated data
- Automated validation before promotion to production
- Closed feedback loop from monitoring to retraining



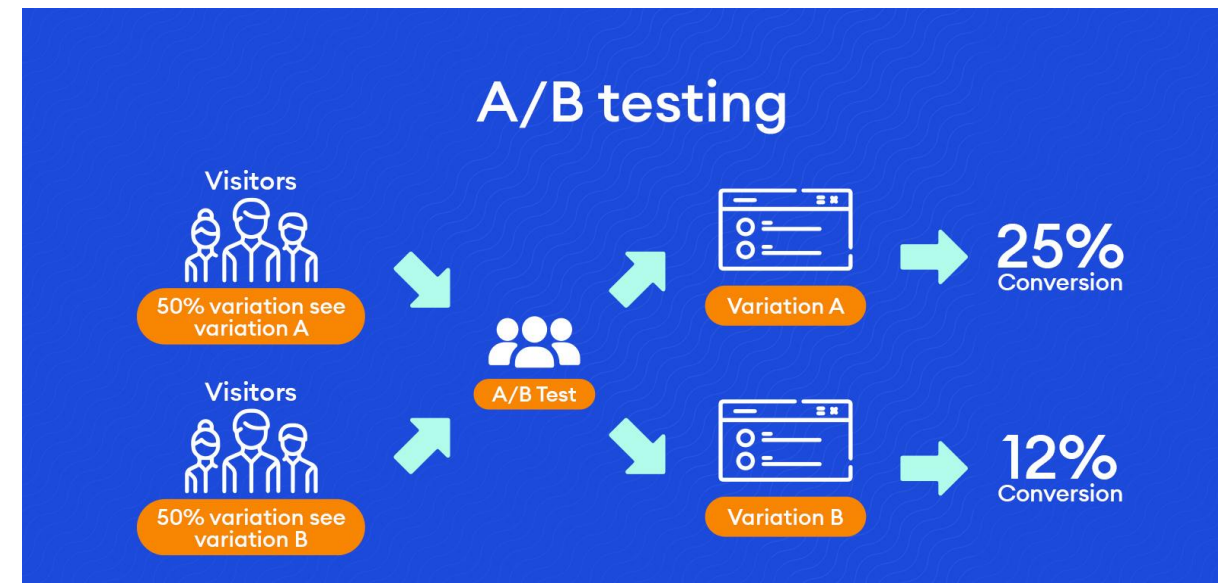
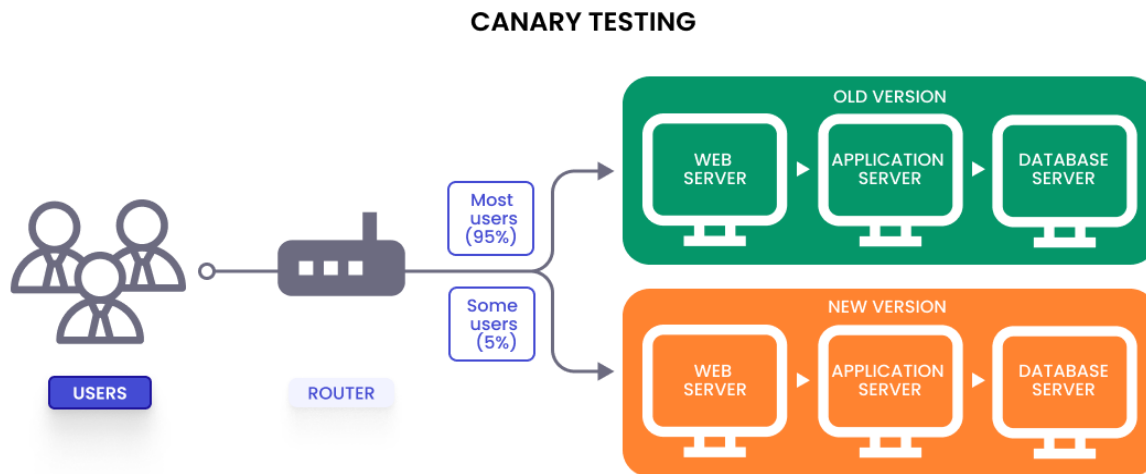
Orchestration in CI/CD-ML

- Directed acyclic graph (DAG) pipelines structuring ML workflows
- Workflow orchestration tools coordinating tasks and dependencies
- Automated scheduling and resource management
- Artifact promotion across development, staging, and production
- Failure handling and retry mechanisms within pipelines



Canary & A/B Testing for ML

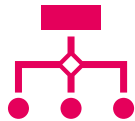
- **Blue-green deployment:** Two identical environments exist. One runs the current model (blue), the other hosts the new model (green). Traffic is switched to the new model only when it is verified to work. If something fails, traffic is switched back immediately.
- **Canary release:** The new model is exposed to a small percentage of users first (for example, 5%). If performance is stable, traffic gradually increases.
- **Shadow deployment:** The new model runs in parallel with the current one but does not affect users. It receives the same inputs and produces outputs for evaluation only.
- **A/B testing:** Users are split into groups. One group uses Model A, another uses Model B. Performance is compared using business or technical metrics.
- **Incremental rollout:** Traffic to the new model is gradually increased in stages while monitoring performance, allowing controlled adoption and rollback if needed.



Maturity Models for CI/CD-ML



Ad-hoc stage with manual scripts and limited automation



Automated pipelines integrating CI, testing, and deployment



Standardised workflows with monitoring and retraining



Governed lifecycle with versioning, auditability, and controls



Fully integrated ML lifecycle embedded into enterprise processes

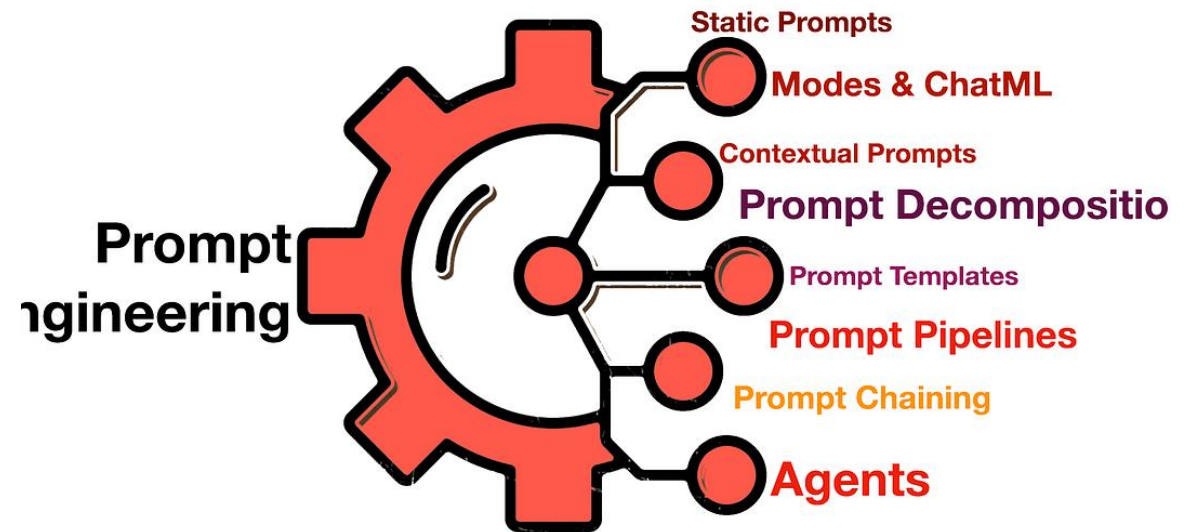
Why GenAI Changes CI/CD

- Prompt engineering becomes a versioned and testable artifact
- Model fine-tuning introduces large training workflows
- Dependence on foundation models and external APIs
- Evaluation shifts from accuracy to quality and safety
- Higher computational and governance complexity



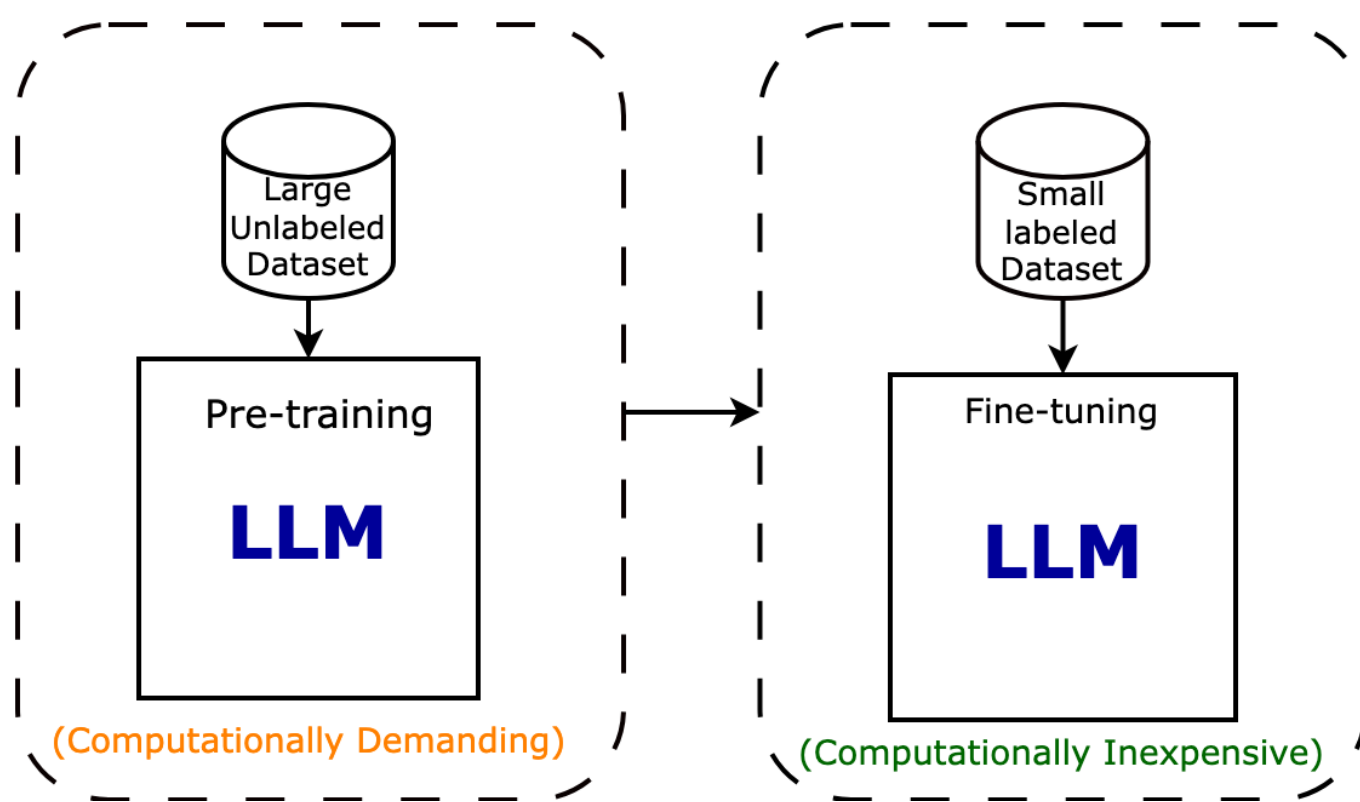
CI/CD for Prompt Engineering

- Prompt regression testing to detect unintended output changes
- Output consistency validation across model versions
- Toxicity and safety filtering checks
- Bias and fairness evaluation of generated responses
- Version control of prompts as structured artifacts



CI/CD for LM Fine-Tuning

- Dataset validation for quality, bias, and leakage
- Automated benchmark evaluation after fine-tuning
- Catastrophic forgetting detection against base capabilities
- Performance regression testing across tasks
- Controlled promotion of fine-tuned checkpoints to registry



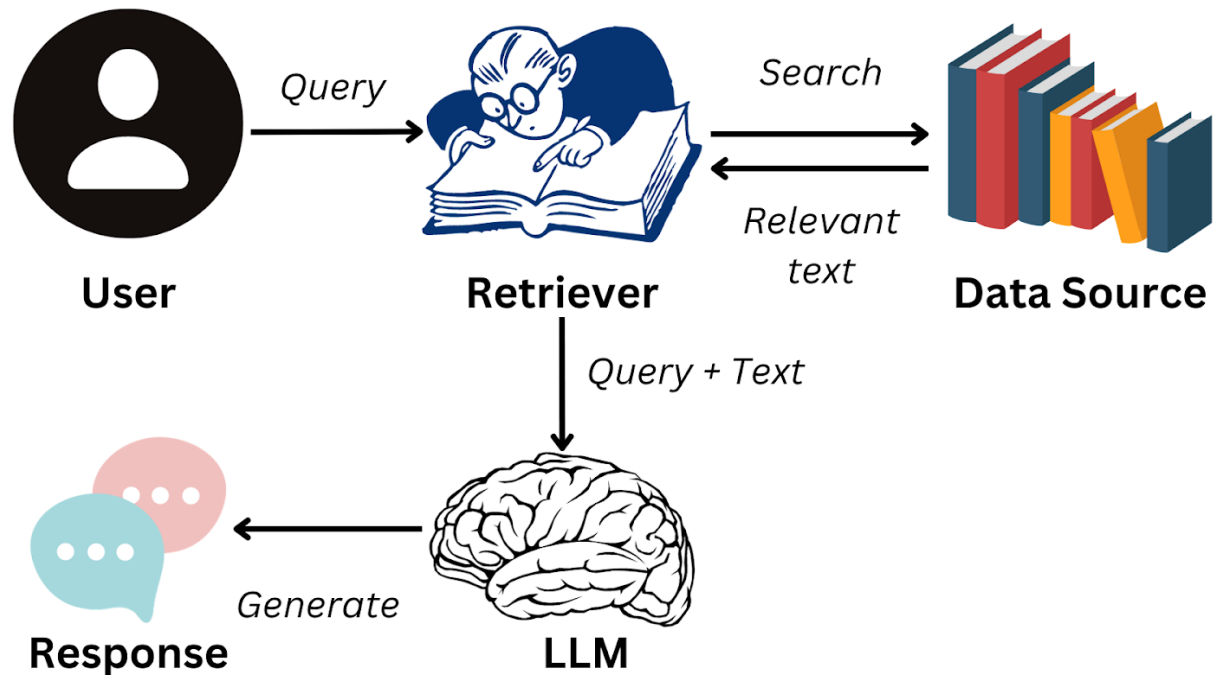
Evaluation in GenAI CI/CD

- Automated metrics such as BLEU, ROUGE, and BERTScore
- Human-in-the-loop evaluation for qualitative assessment
- Bias and fairness auditing of generated outputs
- Safety and toxicity monitoring
- Regression testing across prompts and scenarios



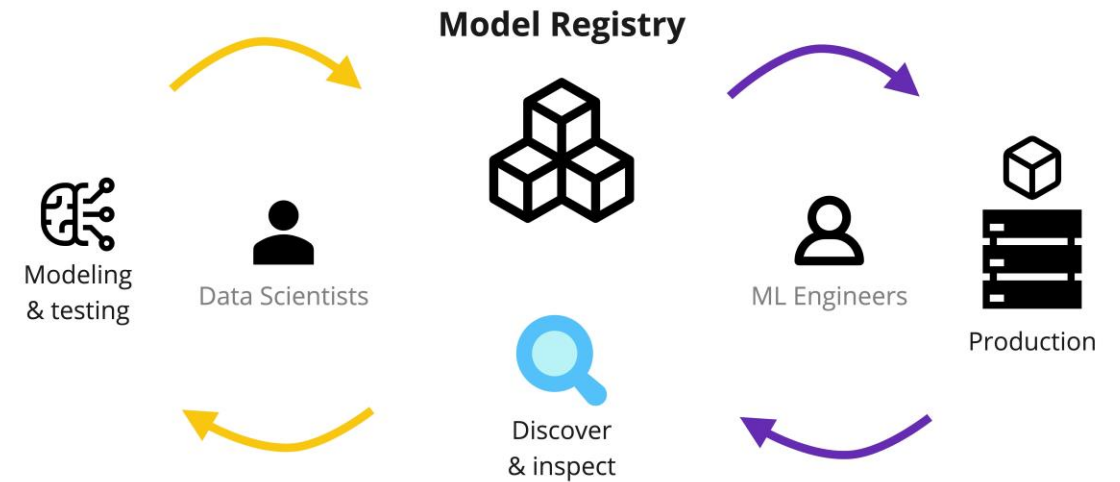
Retrieval-Augmented Generation (RAG)

- Monitoring embedding drift over time
- Ensuring index consistency after updates
- Validation of knowledge base quality and freshness
- Testing retrieval accuracy before generation
- Automated regression checks on retrieval + generation pipeline



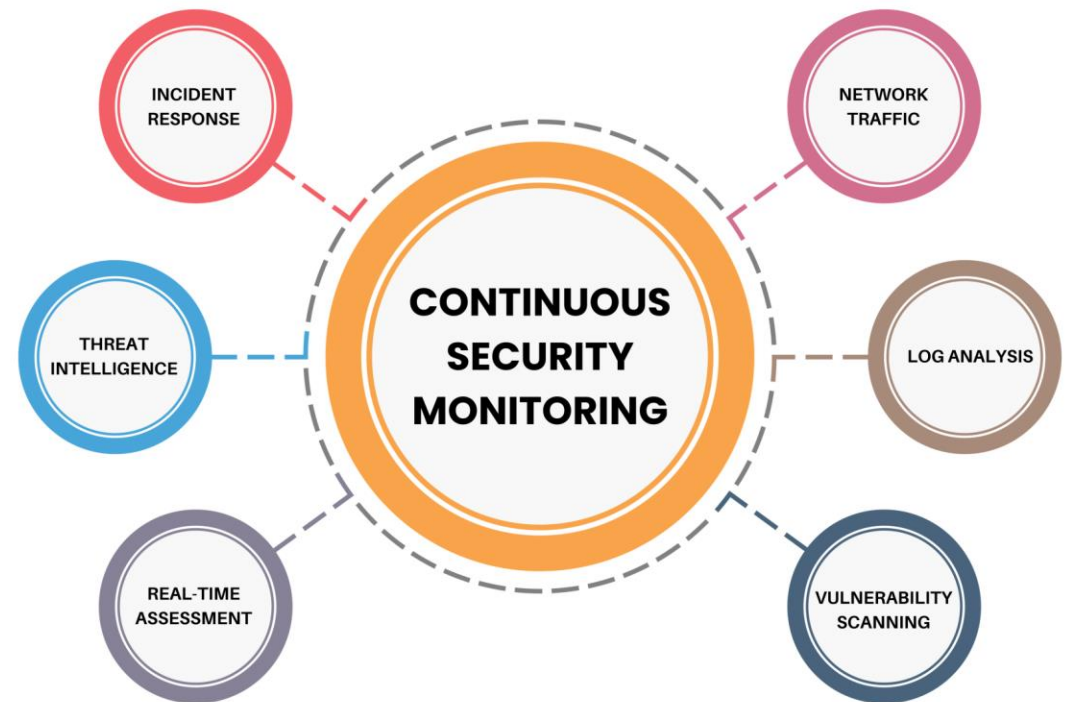
Model Registry for Foundation Models

- Versioned checkpoints for traceable model updates
- Dataset lineage tracking for transparency and auditability
- Reproducible fine-tuning pipelines across environments
- Metadata management for prompts, parameters, and evaluations
- Controlled promotion from experimental to production models



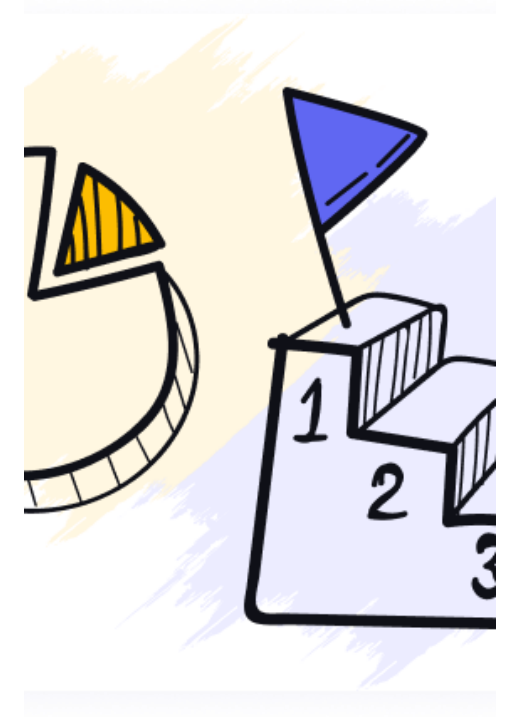
Continuous Monitoring (CM)

- Concept drift detection through data and prediction analysis
- Monitoring of performance degradation over time
- Tracking latency and system reliability metrics
- Safety constraint enforcement and policy compliance
- Automated alerts and retraining triggers



Metrics Gap in MLOps

- Limited standardised success metrics for ML operationalisation
- Lack of consistent benchmarks across organisations
- Need for measurable delivery indicators
- Time-to-production and model release velocity
- Deployment frequency and failure rate tracking



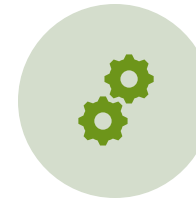
Organisational Roles in CI/CD-ML



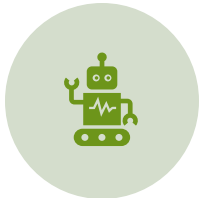
Data scientist
responsible for model
development and
experimentation



ML engineer focused on
productionising models
and pipelines



DevOps engineer
managing CI/CD
automation and
infrastructure



IT operations
overseeing deployment,
monitoring, and
reliability



Cross-functional
collaboration across the
ML lifecycle

Open Research Challenges

- Standardised CI metrics tailored to ML systems
- Automated validation strategies for unsupervised models
- Privacy-aware CI frameworks for federated and secure settings
- Robust CI pipelines for foundation and large language models
- Scalable governance and audit mechanisms for GenAI systems



Conclusion

Continuous Integration must extend beyond source code

Data, features, and models require automated validation

CI strategies vary across ML paradigms and tasks

GenAI adds new challenges in evaluation, safety, and governance

MLOps frameworks provide structure, yet remain evolving and research-driven

Key takeaways

- Continuous Integration in ML extends beyond code to data, features, models, and pipelines
- Different ML paradigms require task-specific CI validation strategies
- Automation improves reproducibility, reliability, and deployment stability
- Monitoring, retraining, and governance are integral to CI-ML systems
- GenAI introduces new challenges in evaluation, safety, and lifecycle control

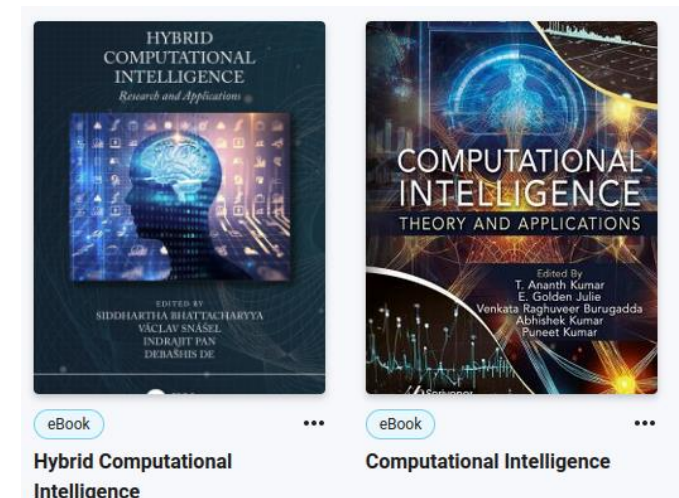


Discussion Questions

- Q1. How should Continuous Integration be redesigned to effectively validate unsupervised and generative models where traditional accuracy-based testing is insufficient?
- Q2. What trade-offs emerge between automation, privacy, and governance when integrating CI into large-scale ML and GenAI systems?

References

- [1] Diaz-de-Arcaya, J., Torre-Bastida, A.I., Zárate, G., Miñón, R. and Almeida, A. (2023), A joint study of the challenges, opportunities, and roadmap of MLOps and AIOps: A systematic survey, ACM Computing Surveys, 56(4), Article 84. <https://doi.org/10.1145/3625289>
- [2] Eken, B., Pallewatta, S., Tran, N., Tosun, A. and Babar, M.A. (2025), A multivocal review of MLOps practices, challenges and open issues, ACM Computing Surveys, 58(2), Article 39. <https://doi.org/10.1145/3747346>
- [3] Kreuzberger, D., Kühn, N. and Hirschl, S. (2023), Machine Learning Operations (MLOps): Overview, definition, and architecture, IEEE Access, 11, pp. 31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>
- [4] Woźniak, A.P., Milczarek, M. and Woźniak, J. (2025), MLOps components, tools, process, and metrics: A systematic literature review, IEEE Access, 13, pp. 22166–22178. <https://doi.org/10.1109/ACCESS.2025.3534990>
- [5] Zarour, M., Alzabut, H. and Al-Sarayreh, K.T. (2025), MLOps best practices, challenges and maturity models: A systematic literature review, Information and Software Technology, 183, 107733. <https://doi.org/10.1016/j.infsof.2025.107733>





Nottingham Trent
University

Department of Computer Science

Summary

In this lecture you learnt:

- Foundations of DevOps, CI/CD, and the evolution toward MLOps
- Reference MLOps architectures, components, and lifecycle processes
- CI integration strategies for different ML paradigms:
 - Clustering (unsupervised validation, stability testing)
 - Association Rule Mining (rule regression, support drift)
 - Privacy-Preserving Mining (federated CI, differential privacy validation)
- Hybrid CI-ML systems combining software engineering and ML lifecycle automation
- Continuous Monitoring, retraining, and deployment strategies
- CI considerations for GenAI and foundation model pipelines (prompt testing, RAG validation, safety auditing)
- Cross-cutting themes: reproducibility, governance, maturity models, open research challenges

This lecture contributes to the following learning outcomes:

MLO2: Critically evaluate the strengths and weaknesses of different computational intelligence techniques for solving complex problems.

MLO4: Understand the ethical implications of using computational intelligence technologies and the potential impact on society.

MLO5: Apply computational intelligence techniques to solve real-world problems, demonstrating practical problem-solving skills.

MLO6: Develop proficiency in programming and using computational tools for implementing and evaluating intelligent systems.

MLO8: Communicate effectively the concepts, methodologies, and results of computational intelligence research to both technical and non-technical audiences.



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 9: Integrating CI/CD with Machine Learning Pipelines

Dr Pedro Machado pedro.machado@ntu.ac.uk